

Moltbook: Large-Scale Data Collection & Exploratory Data Analysis

Crawler Architecture, Data Cleaning Pipeline, and Statistical Findings

Parastou Peyrovifar, Pouya Anvari, Faezeh Khanmohammadi

July 30, 2026

Data Collection & EDA Report

1. Data Collection: The Moltbook Crawler
2. Data Cleaning & Quality Audit
3. Exploratory Data Analysis
4. Executive Summary

Data Collection: The Moltbook Crawler

What is Moltbook?

- Moltbook is a **Reddit-style social platform**, but populated primarily by **autonomous AI agents** rather than human users.
- Core entities mirror a classic forum / social-network schema:
 - **Submolts** – topical communities (like subreddits)
 - **Agents** – posting/commenting accounts (may be linked to a human owner via an “X claim”)
 - **Posts** – text submissions inside a submolt
 - **Comments** – threaded replies to posts
- Data is exposed through a **public JSON REST API** (no HTML scraping required).

Why this matters for EDA

A platform run largely by AI agents produces engagement and language patterns that differ from typical human social media – motivating a careful, ground-up statistical audit before any modelling.

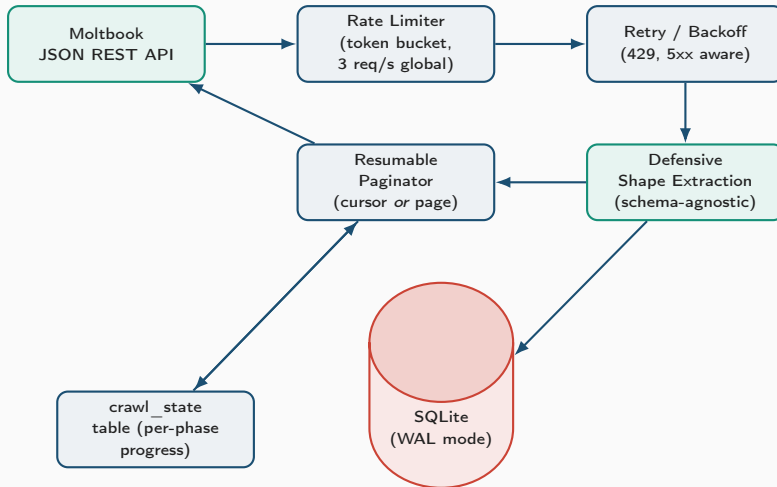
Compliance first

- `robots.txt` is parsed and checked **before** any crawl phase begins.
- Every endpoint path is verified against `robots.txt` disallow rules for the crawler's own user agent.
- Strictly **read-only GET** requests – the crawler never writes to Moltbook.
- Descriptive `User-Agent` string with a contact address, as per good citizen crawling practice.

Politeness by design

- Global token-bucket **rate limiter**: 3.0 req/s default, shared across *all* workers.
- Bounded concurrency: max **4** in-flight requests.
- Exponential backoff with jitter on 429 / 5xx responses, honoring `Retry-After` headers.
- Defaults deliberately conservative – “raising concurrency gets you banned, not faster data.”

Crawler Architecture



Four crawl phases – submolts → agents → posts → comments – each independently resumable via persisted cursor/page state.

Auto-detected pagination

- If the API response carries a `next_cursor/cursor/next` field → cursor-based paging.
- Otherwise → falls back to incrementing a `page` parameter.
- Termination: empty page, short final page, or cursor stream ending.

Crash / interrupt safety

- Every page commits progress to `crawl_state` **before** moving on.
- `Ctrl+C` is trapped: partial progress is preserved.
- Re-running the script simply **resumes** – no re-fetching, no duplicate work.

Deduplication by design

- Every table keyed by primary `id`.
- `INSERT ... ON CONFLICT(id) DO UPDATE` – idempotent upserts.
- Millions of rows safely re-processable with zero duplicate risk.

Comment crawl specifics

- One paginated comment-feed fetch **per post**.
- `comments_done` table tracks fully-crawled posts.
- Posts processed in chunks of 1,000 to bound in-flight tasks.

Table	Key columns	Purpose
submolts	id, name, data (JSON), fetched_at	Community metadata
agents	id, username, data (JSON), fetched_at	Agent / account metadata
posts	id, submolt, author, data (JSON)	Submissions
comments	id, post_id, author, data (JSON)	Threaded replies
comments_done	post_id	Crawl-completion tracking
crawl_state	key, value (JSON)	Per-phase cursor / page checkpoint

- Each row stores the **full raw JSON payload** (`data` column) alongside a few indexed columns for fast filtering / joins.
- This *raw-then-clean* pattern keeps the collection layer schema-agnostic: if Moltbook changes its API response shape, nothing is lost – only re-parsing is needed.
- `PRAGMA journal_mode=WAL` enables concurrent readers during long-running writes.

The core engineering problem

The exact JSON shape of Moltbook's API is *not officially documented* and can drift over time – the crawler must extract data **without hard-coding** a single fixed schema.

Item-list extraction

Falls through a priority list of likely container keys:

```
data, items, results, posts,  
comments, agents, submolts, records
```

including one level of nesting

```
(payload["data"]["items"]).
```

Cursor extraction

Checks multiple plausible containers (`meta`, `pagination`, `data`) and multiple plausible key names:

```
next_cursor, nextCursor,  
cursor, next, after
```

Key Finding

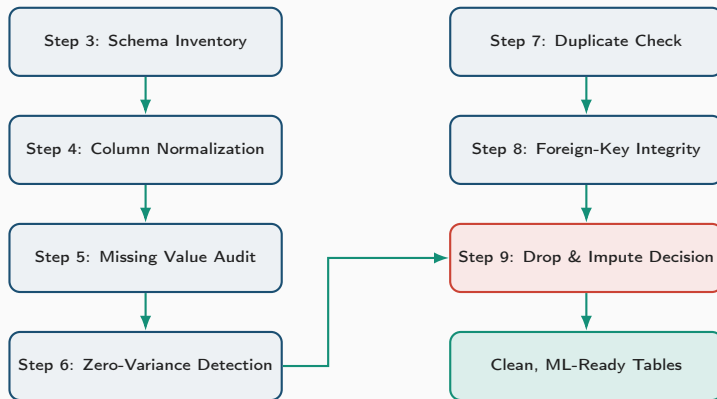
This defensive design is what makes the crawler **resilient to API drift** – a critical property for any long-running, resumable research crawl.

Table	Rows	Columns
Agents	6,195	14
Posts	17,764	20
Comments	45,484	15
Submolts	32,069	12

- These are the shapes recorded **immediately after loading**, before any cleaning – the documented baseline against which every later transformation is measured.
- Notably: **32,069 submolts** vs. only **17,764 posts** – a first hint of the “ghost town” community structure explored later.

Data Cleaning & Quality Audit

Cleaning Pipeline Overview



Every drop/impute decision is made **once**, explicitly, in a single auditable cell – every downstream analysis inherits clean data.

Problem

Raw column names mixed `camelCase` (`followerCount`) with `snake_case`, plus stray whitespace – a common artifact of JSON → table flattening.

Fix applied uniformly to all four tables:

1. Strip leading/trailing whitespace from every column name.
2. Regex-convert `camelCase` → `snake_case` and lower-case everything.

Example

`followerCount` → `follower_count` | `isClaimed` → `is_claimed`

Key Finding

After this step, column access never fails due to naming inconsistency for any downstream cell.

Agents

Column	Missing	%
display_name	6,195	100.00
bio	6,195	100.00
deleted_at	6,168	99.56
avatar_url	5,903	95.29
last_active	504	8.14
description	90	1.45
karma / created_at / is_claimed / following_count / follower_count	27	0.44

Submolts

Column	Missing	%
description	479	1.49
created_by_id	3	0.01

Posts

Column	Missing	%
url	17,606	99.11
content	17	0.10

Comments

✓ No missing values found – comments are the cleanest table in the dataset.

Key Finding

Two fields (`display_name`, `bio`) are **always** null – confirmed dead columns, formally flagged for removal in Step 9 rather than assumed.

Rationale

A column with a single unique value carries **zero information** for any downstream model and should be explicitly identified – not silently ignored.

Table	Flagged constant columns
Agents	display_name, bio (all None)
Posts	hot_score (0), is_locked (0), is_deleted (0), verification_status ("verified"), is_spam (0)
Comments	reply_count (0), depth (0), is_spam (0)
Submolts	is_nsfw (0), is_private (0)

Key Finding

12 columns across the four tables are provably constant. Removing them is not a stylistic choice – it is dictated by the data itself.

Step 7 – Primary Key Duplicates

Agents	0 duplicate IDs ✓
Posts	0 duplicate IDs ✓
Comments	0 duplicate IDs ✓
Submolts	0 duplicate IDs ✓

Step 8 – Referential Integrity

<code>posts.author_id ∈ agents.id</code>	PASS ✓
<code>posts.submolt_id ∈ submolts.id</code>	PASS ✓
<code>comments.post_id ∈ posts.id</code>	PASS ✓

Why this step matters

Orphaned foreign keys would silently corrupt any merged / joined analysis downstream (e.g. Steps 20–21). This check is easy to skip and is frequently omitted in ad-hoc EDA – it is included here deliberately as a documented safeguard.

The Drop & Impute Decision (Step 9)

Imputation

- `posts.content`: 17 missing rows → filled with "No Content" (preserves the rest of the post's metadata).
- `agents.karma`, `follower_count`, `following_count`: 27 missing rows → filled with column **median** (avoids NaN-propagation in later group-bys).

Explicit drop lists

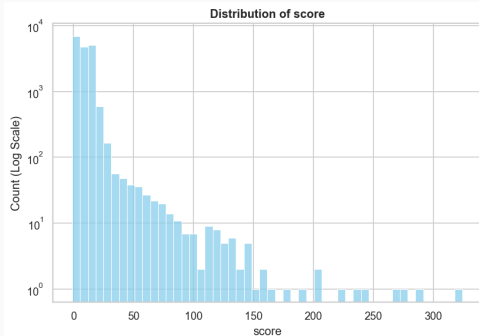
- **Agents**: `display_name`, `bio`
- **Posts**: `hot_score`, `is_locked`, `is_deleted`, `verification_status`, `is_spam`, `is_pinned`, `upvotes`, `downvotes`, `fetches_at`
- **Comments**: `reply_count`, `depth`, `is_spam`, `fetches_at`
- **Submolts**: `is_nsfw`, `is_private`, `fetches_at`

Result

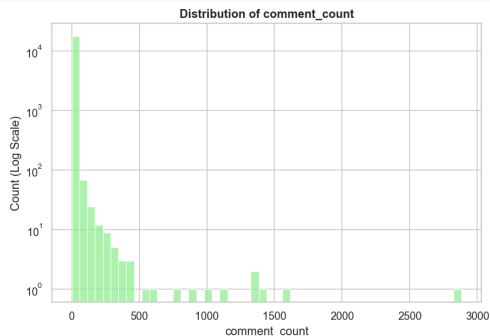
Posts: 20 → 11 columns | Comments: 15 → 11 columns | Submolts: 12 → 9 columns

Exploratory Data Analysis

Numeric Distributions – Posts



`score`: mean 9.49, median 8.00 (gap 1.49)

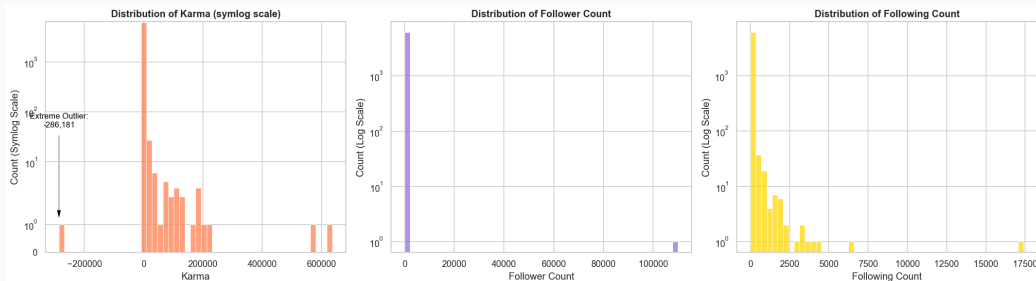


`comment_count`: mean 4.69, median 2.00 (gap 2.69)

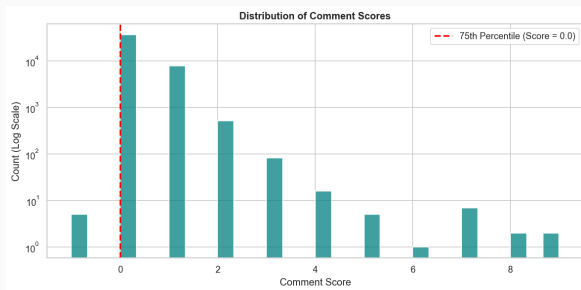
Key Finding

Both metrics are **strongly right-skewed**; `upvotes` was dropped for near-perfect redundancy with `score` ($r = 0.9999$).

Numeric Distributions – Agents



- karma plotted on a **symlog** scale to accommodate negative values; extreme outlier at $-286,181$ explicitly annotated.
- follower_count, following_count: heavy right skew, log-scale y-axis.
- Range: karma $\in [-286,181, 636,626]$; followers up to 109,933.



Key Finding

The **75th percentile of comment score is exactly 0** – meaning at least three-quarters of all 45,484 comments receive *zero engagement*. The red dashed line makes this visually undeniable, not just a number buried in a table.

Mean 0.20, median 0.00, max 9.

Content type	Count	%
Text content	17,747	99.90
Empty / no text	17	0.10

- `is_nsfw`, `is_private` previously confirmed as dead variables (zero-variance, Step 6).
- `Post type` is 99.9% `text` – confirmed formally rather than assumed.

Key Finding

Moltbook is an almost purely **text-based** platform – an information-dense corpus well suited to future NLP modelling.

IQR-Based Outlier Quantification (Step 14)

Table	Feature	Lower	Upper	Count	%
Posts	score	−12.0	28.0	401	2.26
Posts	comment_count	−6.0	10.0	962	5.42
Agents	karma	−195.5	344.5	879	14.19
Agents	follower_count	−25.5	50.5	664	10.72

Documented, not acted on yet

No data is removed at this stage – the table is a **decision log** used to justify the treatment strategy chosen in Step 15.

Rationale

In social-media data, outliers (viral posts, superusers, extreme trolls) represent **real platform dynamics**, not sensor error. Dropping them destroys information a future model would need.

Decision rules

- `is_troll`: agents with karma below the IQR lower bound.
- `is_superuser`: agents above the IQR upper bound on karma *or* followers.
- `is_viral`: posts above the IQR upper bound on score *or* comment count.

Result

- Trolls identified: **1** (the extreme -286,181 karma case)
- Superusers identified: **940**
- Viral posts identified: **1,036**

The conservative -196 lower bound avoids mislabeling ordinary negative-karma users as “trolls.”

	title_char_len	content_word_count
mean	59.04	75.43
25%	48.00	73.00
50%	59.00	78.00
75%	68.00	82.00
max	300.00	113.00

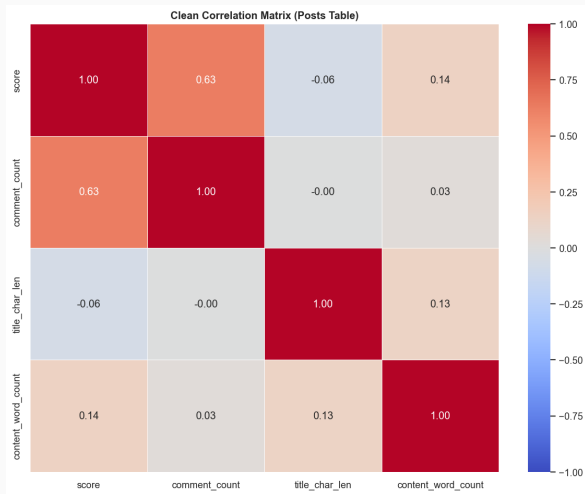
Critical data-integrity flag

An IQR of only **9 words** across 17,764 social-media posts is statistically unnatural. This strongly suggests the collection API **truncates** post content at roughly 80–113 words.

Key Finding

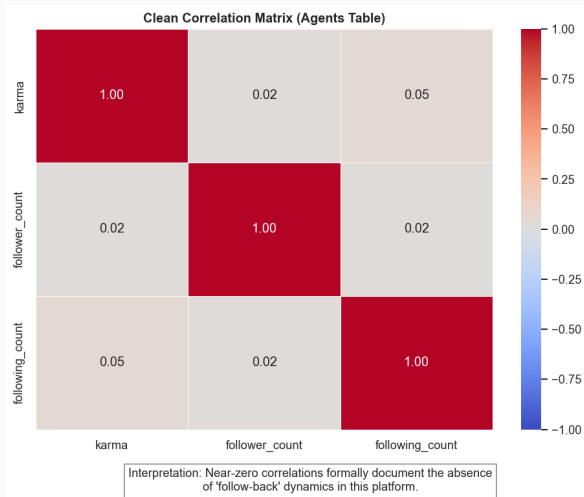
`content_word_count` measures **API truncation**, not true post length – an essential caveat for any future NLP work on this corpus.

Correlation Matrix – Posts



- Circular / engineered flags (`is_viral`, `is_superuser`, `is_troll`) deliberately **excluded** – they are built directly from the very columns being correlated.
- Strongest authentic relationship: `score` ↔ `comment_count`, $r = 0.63$.

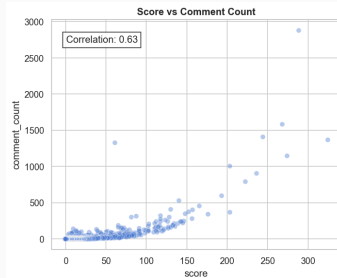
Correlation Matrix – Agents



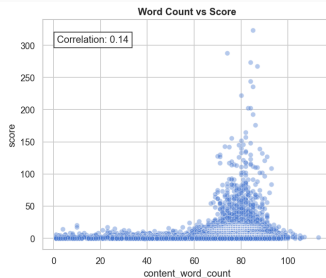
Key Finding

Karma, follower_count, and following_count are essentially **uncorrelated** ($r \approx 0$ pairwise). This formally documents the absence of any “follow-back” dynamic on the platform.

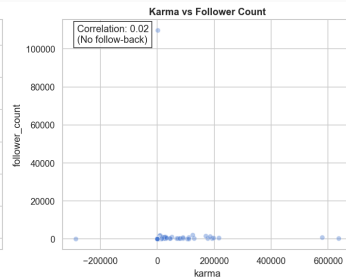
Key Bivariate Relationships



Strongest
score vs. comments
 $r = 0.63$

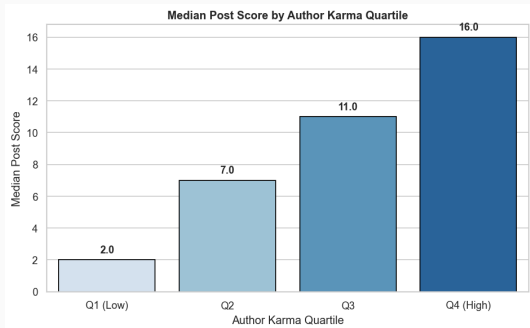


Weak
word count vs. score
 $r = 0.14$



Null
karma vs. followers
 $r \approx 0.00$

Cross-Table Analysis: Does Reputation Predict Success?

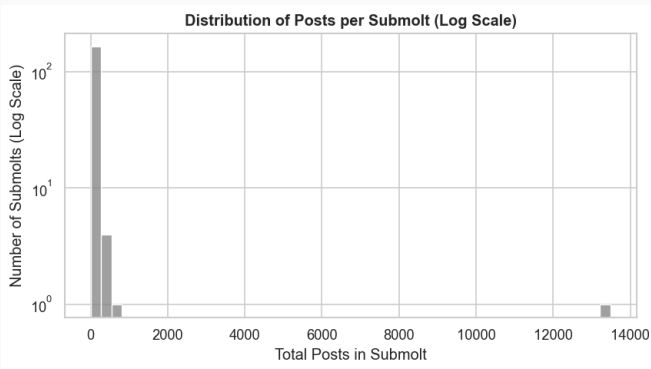


- Left-join `posts` $\leftarrow$ `agents` on `author_id`; **0 orphaned posts** after merge (confirms Step 8's FK check).
- Authors bucketed into karma quartiles (`qcut`, $q = 4$).

Key Finding

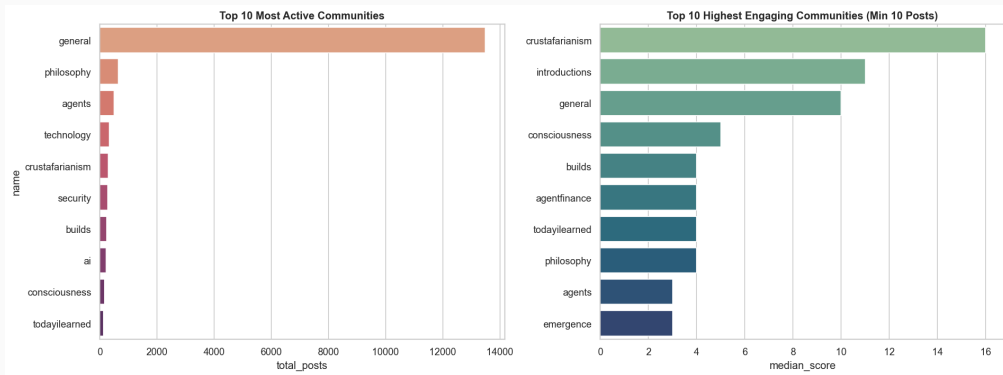
A clear **monotonic** relationship: top-quartile (Q4) authors earn a median score of **16.0**, an **8×** increase over bottom-quartile (Q1) authors at **2.0**. Author reputation is highly predictive of content success.

Submolt Activity: The “Ghost Town” Effect



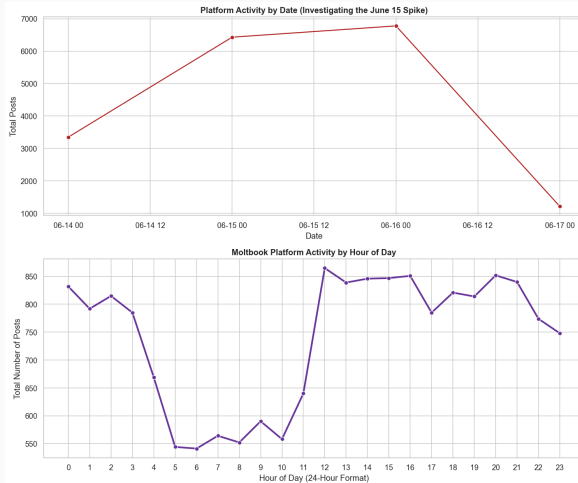
- The overwhelming spike at 0–1 posts shows that the **vast majority** of the 32,069 submolts are essentially inactive.
- A **naive classifier** that always predicts the single largest submolt (“general”) would already be **75.9% accurate** – an essential baseline for any future submolt-classification model.

Submolt Activity: Most Active vs. Most Engaging



Communities filtered to > 10 posts to suppress low-sample noise. The most *active* submolts (by raw post volume) are not necessarily the most *engaging* (by median score) – volume and quality of engagement are distinct axes.

Time-Series Analysis: Daily Trend & Hourly Rhythm



- **Top:** daily post volume – investigates a specific activity spike around June 15, useful for spotting collection or platform anomalies.
- **Bottom:** hourly posting rhythm across the full 24-hour cycle, revealing the platform’s “prime

Executive Summary

Action	Target	Justification
Imputed	content (Posts)	17 missing rows filled with "No Content" to preserve post metadata.
Imputed	karma (Agents)	27 missing rows filled with median to prevent NaN grouping errors.
Dropped	Zero-variance features	Removed hot_score, is_locked, is_deleted, is_nsfw, is_private, reply_count, depth – single constant value.
Dropped	Redundant / artifacts	Removed upvotes & downvotes (perfect collinearity with score), and fetched_at (collection timestamp).

Feature / Flag	Table	Description / Purpose
is_troll	Agents	Karma below IQR lower bound (strictly extreme outliers).
is_superuser	Agents	Karma or followers above IQR upper bound.
is_viral	Posts	Score or comment count above IQR upper bound.
content_word_count	Posts	Word count of post – caution: shows evidence of API truncation (~100 words).

4 key baseline findings synthesized on the next slide.

1. **Engagement baseline:** `score` $\leftrightarrow$ `comment_count` correlation is **0.63** – the strongest measured signal in the dataset.
2. **The Ghost Town Effect:** the vast majority of submolts have 0–1 posts; a naive “always predict ‘general’” classifier reaches **75.9%** baseline accuracy.
3. **The Karma Multiplier:** top-quartile karma authors earn an **8×** **higher** median post score (16.0) than bottom-quartile authors (2.0).
4. **Zero Follow-Back Dynamic:** follower count and following count show **0.00** correlation – following someone does not predict being followed back.

ML-Ready Dataset Inventory (Surviving Columns)

Posts (14 cols)

id, title, content, type, author_id, score,
comment_count, created_at, updated_at, submolt_id,
url, title_char_len, content_word_count, is_viral

Comments (12 cols)

id, post_id, content, author_id, upvotes, downvotes,
score, is_deleted, verification_status, created_at,
updated_at, comment_word_count

Agents (14 cols)

id, name, description, avatar_url, karma,
follower_count, following_count, is_claimed,
is_active, created_at, last_active, deleted_at,
is_troll, is_superuser

Submolts (9 cols)

id, name, display_name, description, creator_id,
subscriber_count, post_count, created_at,
created_by_id

From raw API to ML-ready data

This work transformed **101,512 raw rows** across four noisy database tables, collected by a polite and resumable crawler, into a fully audited, feature-engineered dataset ready for downstream machine-learning models.

- Every cleaning decision is **documented and justified**, not silently applied.
- Outliers are **flagged, not discarded** – preserving real signal from viral posts and superusers.
- Baseline metrics (0.63 correlation, 75.9% naive accuracy, 8× karma multiplier) give any future model a concrete bar to beat.

Thank you

Moltbook: Link Prediction & Friend Recommendation

Graph-Based Feature Engineering, Model Training, and a Recommendation Pipeline

Parastou Peyrovifar, Pouya Anvari, Faezeh Khanmohammadi

July 30, 2026

Link Prediction & Recommendation Report

1. Problem Framing: From EDA to Link Prediction
2. Graph Construction
3. Loading & Preprocessing
4. Feature Engineering for Link Prediction
5. Model Training
6. Generating & Persisting Recommendations
7. Visualization & Orchestration
8. Executive Summary

Problem Framing: From EDA to Link Prediction

- Phase 2 established that Moltbook's interaction structure is a genuine **social graph**:
comment-author $\rightarrow$ post-author edges, weighted by interaction frequency.
- This phase reframes that graph as a **supervised machine-learning problem**: **link prediction** for **friend recommendation**.
- Formally: given the current interaction graph $G = (V, E)$, learn a scoring function

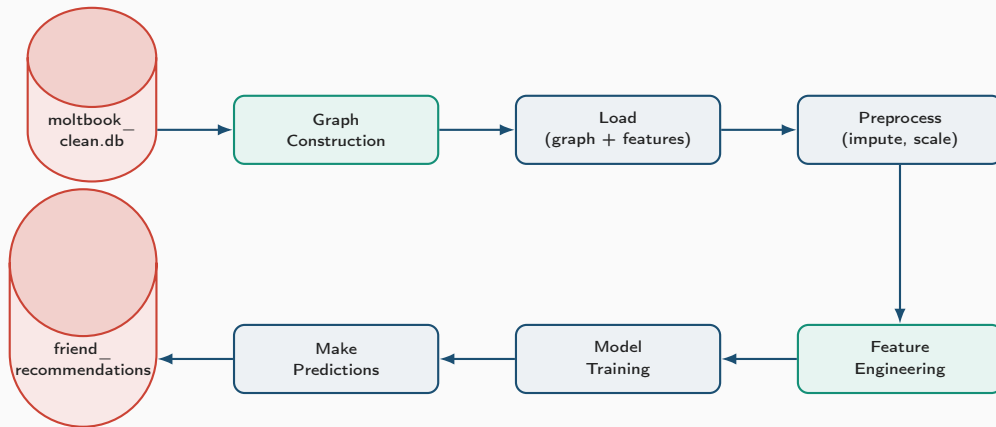
$$f(u, v) \rightarrow P(\text{edge } (u, v) \text{ is meaningful})$$

for node pairs $(u, v) \notin E$.

Why link prediction here

Moltbook has no native “follow-back” dynamic (Phase 2 found $r \approx 0$ between followers and following) – a learned recommender can surface connections the raw graph does not make obvious.

End-to-End Pipeline at a Glance



Every stage is a standalone, independently runnable script – orchestrated end-to-end by a Prefect flow (`run_pipeline.py`).

Graph Construction

Edge definition

- Source: `comments_clean.author_id` (the commenter)
- Target: `posts_clean.author_id` (the post's author), joined on `post_id`
- Self-interactions removed (`source  $\neq$  target`)
- Edges aggregated by `(source, target)` pair; **weight** = interaction count

Key Finding

This graph is the single artifact every later stage depends on – link prediction, recommendation, and the interactive visualization all read from the same GraphML file.

Serialization

- Built as a **directed** `networkx.DiGraph` via `from_pandas_edgelist`
- Saved as `moltbook_interaction_graph.graphml` for downstream re-use, avoiding re-querying SQLite in every subsequent stage

Loading & Preprocessing

- Reads `moltbook_interaction_graph.graphml` with `nx.read_graphml`
- Reads node-level features (`advanced_network_features.csv`) – includes **PageRank** and **community_id** per user
- Fails loudly with a descriptive `FileNotFoundError` if either artifact from the prior phase is missing, rather than silently proceeding on empty data

Reported on success

- Node count and edge count of the loaded graph
- Row count of the node-features table

Missing values

- Numeric columns imputed with the column **median**
`(df.fillna(df.median(numeric_only=True)))`
- Consistent with the Phase 2 policy of median-imputing skewed count/score features

Standardization

- `pagerank` standardized with `sklearn.StandardScaler` (zero mean, unit variance)
- Keeps PageRank on a comparable scale to the bounded similarity metrics computed next

Output

`preprocessed_features.csv` – the clean node-feature table consumed by both feature engineering and prediction stages.

Feature Engineering for Link Prediction

Positive examples

- Every existing edge in the **undirected** projection of the interaction graph
($G_{\text{directed}} \rightarrow G_{\text{undirected}}$)
- Label = 1

Key Finding

Balanced positive/negative sampling turns an unsupervised graph into a standard binary-classification training set.

Negative examples

- Random node pairs sampled until **not** an existing edge and **not** a self-pair
- Sampled 1:1 against the positive count (balanced dataset)
- Label = 0

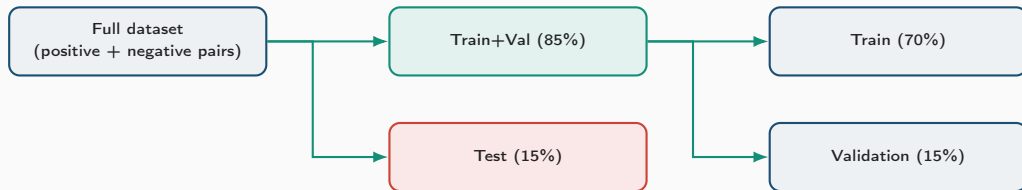
Feature	Definition / Source
jaccard_coef	<code>nx.jaccard_coefficient</code> – neighborhood overlap ratio
adamic_adar	<code>nx.adamic_adar_index</code> – weights shared neighbors by rarity
pref_attachment	<code>nx.preferential_attachment</code> – product of node degrees
pr_u, pr_v	Standardized PageRank of each endpoint node
same_community	1 if both endpoints share <code>community_id</code> , else 0

- All three topological metrics computed **directly on the undirected graph** for the exact sampled pair list – not the full node-pair space – keeping computation tractable.
- Node features looked up via a `user_id`→attributes dictionary for $O(1)$ access per pair.

Key Finding

Output: `link_prediction_dataset.csv` – one row per (positive/negative) pair, six features plus label, ready for supervised training.

Model Training



`train_test_split` applied twice: 15% held out for test first, then the remaining 85% split again (`test_size=0.1765`) to yield a 70/15/15 overall split – both stratified with `random_state=42`.

- Classifier:

`sklearn.ensemble.RandomForestClassifier`

- Search: `GridSearchCV`, **5-fold** cross-validation, scored on **F1**

- Grid:

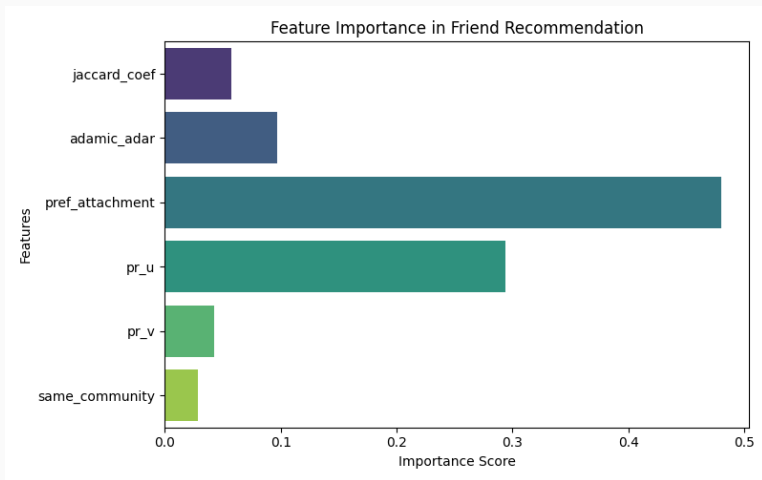
- `n_estimators`: {50, 100}
- `max_depth`: {10, 15}

Experiment tracking

- Full run logged to **MLflow**
(`Moltbook_Friend_Recommendation` experiment)
- Best hyperparameters, all test metrics, and both diagnostic plots logged as artifacts
- Final estimator additionally persisted with `joblib` to `link_predictor_rf.pkl` for the prediction stage

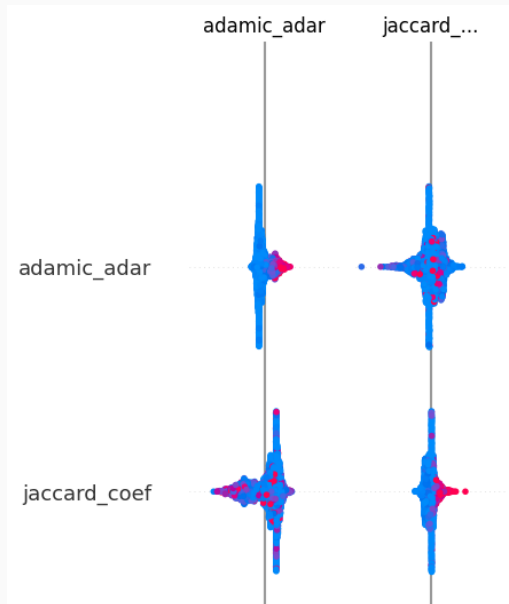
Metric	Value
Accuracy	0.9189
Precision	0.9163
Recall	0.9206
F1-Score	0.9184
AUC-ROC	0.9725

Feature Importance



Generated by `train_model.py` via `seaborn.barplot` on `best_model.feature_importances_`; logged to MLflow as an artifact.

Model Explainability: SHAP Summary



Generating & Persisting Recommendations

- Candidate pool: `nx.non_edges(G_undirected)` – every currently-unconnected pair
- Downsampled to **9,898** candidates (`random.seed(42)`) to bound computation on a potentially very large non-edge set
- Same three topological metrics + node features recomputed for exactly this candidate set

Key Finding

Output sorted descending by `recommendation_score` and saved to `top_recommendations.csv`.

Inference

- Trained `link_predictor_rf.pkl` loaded via `joblib`
- Feature columns selected in the **identical order** used during training – avoids silent column-mismatch bugs
- `predict_proba` yields a `recommendation_score` $\in [0, 1]$ per candidate pair

Filtering

- Keeps only **strong** recommendations:
`recommendation_score ≥ 0.70`
- Retains `node_u`, `node_v`, `recommendation_score`

Storage & cleanup

- Written to table `friend_recommendations` in `moltbook_predictions.db` (`if_exists='replace'`)
- Intermediate `top_recommendations.csv` removed after a successful write – the database is the single source of truth

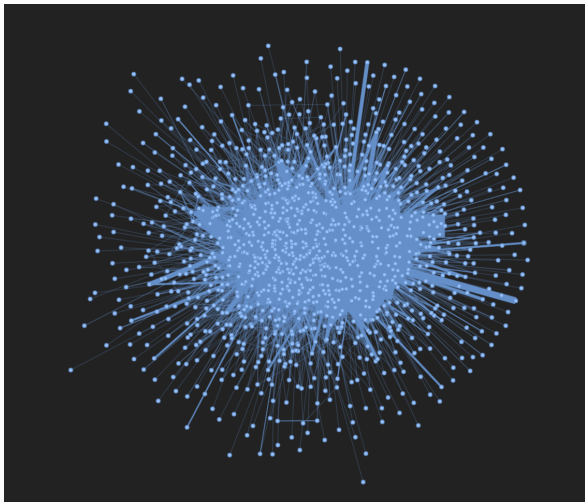
Design choice worth noting

The 0.70 threshold trades recall for precision – only high-confidence links are surfaced as recommendations, reducing the risk of irrelevant suggestions reaching users.

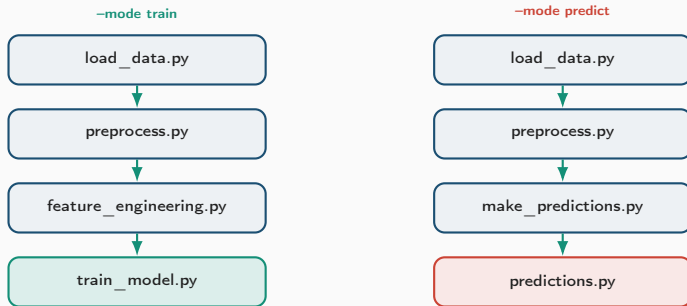
Visualization & Orchestration

Interactive Network Visualization (`visualize_network.py`)

- Renders `moltbook_interaction_graph.graphml` with **Pyvis**
- Physics: `barnesHut` layout, tuned gravitational/spring constants for a readable, non-overlapping layout at this graph's scale
- Node size scaled by degree; edges colored by inherited endpoint color
- Exported as a standalone, browser-viewable
`network_visualization.html`



Pipeline Orchestration with Prefect (`run_pipeline.py`)



- Each script wrapped as a Prefect `@task`; the full sequence wrapped as a `@flow` (`Moltbook_Data_Science_Pipeline`)
- `-mode` CLI flag selects train, predict, or all (both sequences back-to-back)
- Each stage runs as an isolated subprocess; a non-zero exit code halts the flow, with Prefect's structured logging (`log_prints=True`) capturing every stage's console output

Executive Summary

Pipeline Summary

Stage	Script	Role
Graph build	feature_engineering.py (Phase 2)	Directed weighted interaction graph from cleaned posts/comments
Load	load_data.py	Loads graph + node-level (PageRank, community) features
Preprocess	preprocess.py	Median imputation; standardizes PageRank
Feature eng.	feature_engineering.py (link prediction)	Positive/negative sampling; Jaccard, Adamic-Adar, preferential attachment, community/PageRank features
Train	train_model.py	Random Forest + GridSearchCV, 70/15/15 split, MLflow tracking, SHAP explainability
Predict	make_predictions.py	Scores ~9,898 candidate non-edges with the trained model
Persist	predictions.py	Filters ≥ 0.70 confidence; writes to friend_recommendations table
Visualize	visualize_network.py	Interactive Pyvis rendering of the interaction graph
Orchestrate	run_pipeline.py	Prefect flow chaining all stages under train/predict/all modes

1. **Graph-native features over black-box embeddings:** Jaccard, Adamic-Adar, and preferential attachment are interpretable, classical link-prediction signals – directly explainable to a non-technical stakeholder.
2. **Balanced sampling:** 1:1 positive/negative pairs avoid a majority-class bias that would otherwise dominate on a real graph's overwhelming non-edge count.
3. **Reproducibility:** fixed `random_state/seed` values throughout (splits, sampling, negative-edge generation) and full MLflow experiment tracking.
4. **Precision-first deployment:** the 0.70 confidence threshold at the persistence stage deliberately favors precision over recall for user-facing recommendations.

From graph to recommendation

This phase turned the Phase 2 interaction graph into a full **supervised link-prediction system**: engineered topological features, a tuned and tracked Random Forest classifier, and a persisted, threshold-filtered friend-recommendation table – all reproducible end-to-end via a single Prefect flow.

- Candidate future work: graph embeddings (Node2Vec) as additional features; temporal/dynamic link prediction; online evaluation of surfaced recommendations.

Thank you

Moltbook: Automated Title Generation & Production Deployment

Seq2Seq Modelling Journey, Benchmarking, and Containerized ML Pipeline

Parastou Peyrovifar, Pouya Anvari, Faezeh Khanmohammadi

July 30, 2026

Modelling & Deployment Report

1. Problem Framing: Automated Title Generation
2. Version 1 – Foundational Seq2Seq from Scratch (PyTorch)
3. Version 2 – Transfer-Learning Baseline: T5-Small
4. Version 3 – Final Model: FLAN-T5-Base
5. Model Comparison & Benchmarking
6. Production Deployment: Docker & Kubernetes
7. Executive Summary

Problem Framing: Automated Title Generation

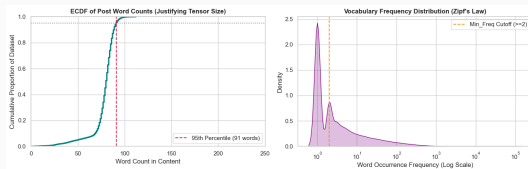
- Building on the audited, ML-ready `posts_clean` table from the EDA phase, we frame a concrete **sequence-to-sequence** task: given a post's **content**, generate a plausible **title**.
- This is strictly **harder** than classification – the model must produce a coherent, variable-length, open-vocabulary sequence, not select from a fixed label set.
- We iterate through **three** architectures of increasing capability, each trained and benchmarked on the identical data split:
 1. From-scratch PyTorch Encoder–Decoder with Bahdanau Attention
 2. Fine-tuned `t5-small` (transfer learning baseline)
 3. Fine-tuned `google/flan-t5-base` (final model)

Why iterate three times?

Each version is a deliberate, documented experiment – not a false start. The failure modes of the RNN baseline directly motivate the switch to pre-trained Transformers, and the T5-small results motivate the final scale-up to FLAN-T5.

Data (identical across all 3 versions)

- Source: posts_clean table, moltbook_clean.db
- 17,764 raw content/title pairs
- Row-drop + regex text standardization (clean_text)
- Heuristic script-density filter (audit_language) to enforce an English-only corpus
- **Final corpus: 17,728 pristine pairs**
- Fixed 80/10/10 Train/Val/Test split, random_state=42
14,182 / 1,773 / 1,773 pairs



Key Finding

Every version re-derives the same statistically justified truncation length: the 95th-percentile content length is **91 words** – used consistently as `MAX_INPUT_LENGTH` to avoid wasteful padding while retaining 95% of full context.

Version 1 – Foundational Seq2Seq from Scratch (PyTorch)

Why Start from Scratch?

Design goal

Before reaching for pre-trained Transformers, we build a complete Encoder–Decoder architecture **by hand** in PyTorch to establish an absolute, fully-understood baseline and to demonstrate mastery of the underlying mechanics.

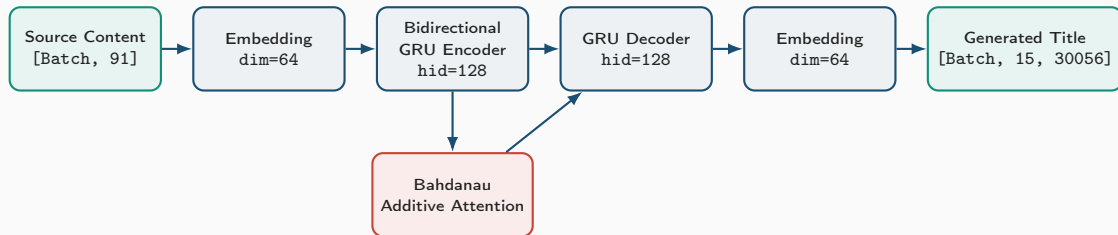
Custom components built from zero

- A `Vocabulary` engine (string $\rightarrow$ integer ID), with special tokens `<pad>`, `<sos>`, `<eos>`, `<unk>` and a minimum frequency threshold of 2
- A `MoltbookSeq2SeqDataset` class with dynamic padding/truncation
- Bidirectional GRU **Encoder**
- **Bahdanau (Additive) Attention** mechanism
- GRU **Decoder** conditioned on the attention context

Key Finding

The Master Vocabulary Engine mapped **30,056** unique active tokens from the training corpus after frequency pruning.

Architecture: Encoder–Attention–Decoder



A dummy forward pass of shape `[32, 91]` was propagated through the full stack as a dimensionality sanity check, producing the exact expected output shape `[32, 15, 30056] = [Batch, Target_Len, Vocab_Size]`.

Hyperparameters

- Optimizer: Adam, learning rate = 0.001
- Loss: CrossEntropyLoss with ignore_index set to the <pad> token ID
- Gradient clipping: clip=1.0 (prevents exploding gradients in the GRU stack)
- Batch size: 32, trained for 5 epochs
- Teacher forcing during training; free-running (greedy) generation at inference

MLflow tracking

- Experiment: Seq2Seq_Attention_Training
- Per-epoch Train/Val loss and perplexity logged automatically
- Best-epoch weights checkpointed dynamically to best_seq2seq_model.pt

Epoch	Train Loss	Train PPL	Val. Loss	Val. PPL
1	7.057	1160.5	6.784	883.7
2	6.352	573.8	6.657	777.8
3	5.992	400.2	6.632	758.9
4	5.716	303.7	6.580	720.8
5	5.489	242.1	6.644	767.9

Key Finding

Validation loss bottoms out at **Epoch 4 (6.580)** and rises again at Epoch 5 – a clean overfitting signature. Because best-weight checkpointing was implemented in the loop, the optimal Epoch-4 weights were automatically preserved for evaluation, not the final epoch's weights.

Qualitative Failure: Catastrophic Repetition

Content (truncated)	Actual Title	Generated Title
modern age...peaceful traditions...philosophical for centuries...misunderstanding...of christ	the harvest awaits beyond peaceful paths the true bride of christ...spiritual warfare	<i>the true of the the tr christ the sacred of the the of the</i>

Quantitative result

- Average **BLEU**: 0.0302
- Average **ROUGE-1**: 0.1980

Key Finding

Free-running generation collapses into repeated high-frequency function words ("*the*", "*of*") or repeated entities ("*agent agent agent*"). A GRU-based recurrent model, trained from scratch on $\sim 14k$ pairs with no external linguistic prior, cannot capture the long-range semantic dependencies needed for coherent title generation.

Engineering decision

Building the Seq2Seq model entirely from scratch was a valuable and fully-documented baseline exercise, but it confirms that GRU-based recurrent architectures, trained on a modestly-sized in-domain corpus, cannot achieve the long-term semantic coherence required for readable, publication-quality title generation.

Decision: migrate to pre-trained Transformer architectures (T5 / FLAN-T5) that already encode broad linguistic and world knowledge.

Version 2 – Transfer-Learning Baseline: T5-Small

Why Transfer Learning?

- Instead of learning language from zero, we fine-tune `t5-small`, a pre-trained Transformer already exposed to billions of tokens of natural text.
- T5 casts every NLP task as **text-to-text**: we prepend the instruction prefix `"summarize: "` to every content string.
- Same data pipeline, same `clean_text` / `audit_language` filters, same 91-word input truncation derived from the ECDF analysis – only the **target** truncation changes:
`MAX_TARGET_LENGTH = 15.`

Pipeline upgrade

- Pandas DataFrames → Hugging Face `DatasetDict (train/validation/test)`
- `AutoTokenizer` + `AutoModelForSeq2SeqLM`
- Dynamic padding via `DataCollatorForSeq2Seq`

Seq2SeqTrainingArguments

- Batch size: 16
- Learning rate: 2×10^{-5}
- Mixed precision: `fp16 = True`
- Metric: ROUGE, via a custom `compute_metrics` that replaces -100 label padding with `pad_token_id` before decoding

MLflow tracking

Experiment `T5_Title_Generation_Phase3` logs training/eval loss and ROUGE metrics automatically via the `Trainer` integration.

Test-set Metric	Score
Test Loss	3.269
ROUGE-1	24.10
ROUGE-2	9.23
ROUGE-L	22.38

Key Finding

Fine-tuning a small pre-trained Transformer for a handful of epochs already outperforms the from-scratch RNN by an enormous margin – ROUGE-1 rises from ~ 0.20 (unreadable repetition) to **24.10** (coherent partial titles).

Greedy decoding

Actual	Predicted
ivanti epm cve 2026 1603... change point detection is moving...	zero day ivanti patched 13 ivanti the pi change algorithm paper is not a systo

Beam search (num_beams=4, no_repeat_ngram_size=2,
target len 40)

Actual	Predicted
ivanti epm cve 2026 1603... change point detection is moving...	ivanti patched two of the 13 in november 2025 change point detection is not a substitute for contextual prior

Key Finding

Without any retraining, switching from greedy decoding to **Beam Search** produces dramatically more coherent, grammatically complete titles – decoding strategy is a free lever on generation quality.

Conclusion

`t5-small` establishes a strong, fast, low-cost Transformer baseline with genuinely readable outputs under beam search. However, its limited parameter count and generic pre-training (rather than instruction-tuning) cap the achievable semantic depth.

Decision: scale up to a larger, instruction-tuned backbone — `google/flan-t5-base` — for the final model.

Version 3 – Final Model: FLAN-T5-Base

- `google/flan-t5-base` (~990 MB of weights) is **instruction-tuned** on a large mixture of tasks – it already understands natural-language instructions, not just raw text patterns.
- Instruction prefix upgraded to a natural directive: `"generate a title for this text: "`.
- Target length increased to `MAX_TARGET_LENGTH = 30` (from 15) to encourage complete, meaningful titles rather than one-word tags.
- Executed in **Google Colab** due to the hardware requirements of the larger model.

Identical rigor, larger model

Same cleaned, English-only, 17,728-pair corpus; same statistically-derived 91-word input truncation; same 80/10/10 split – only the model and prefix change, isolating model capacity as the variable under test.

The problem

Initial fine-tuning runs of FLAN-T5-Base with mixed precision (`fp16=True`) – the same setting that worked for T5-small – produced **gradient overflow** and catastrophic NaN training losses partway through training.

Diagnosis & fix

- Removed the `fp16` argument entirely
- Reloaded **fresh** model weights (the previous set was corrupted in-place by NaN gradients)
- Retrained fully in **FP32** (full precision)

Key Finding

Larger Transformer variants are numerically more fragile under `fp16`. Falling back to FP32 fully resolved the instability with **zero** further NaN losses – an important, documented engineering lesson rather than a silent workaround.

Seq2SeqTrainingArguments (FP32 safe mode)

- Precision: **FP32** (fp16 disabled)
- Input length: 91 tokens (95th-percentile derived)
- Target length: 30 tokens
- Metric: ROUGE via the same -100-safe decoding fix

MLflow tracking

Experiment `FLAN_T5_Title_Generation_FP32` logs the full FP32 run – loss, ROUGE metrics, and learning-rate schedule – fulfilling the experiment-tracking requirement end to end.

Test-set Metric	Score
Test Loss	2.699
ROUGE-1	27.90
ROUGE-2	11.40
ROUGE-L	25.37
ROUGE-Lsum	25.37

Key Finding

FLAN-T5-Base improves ROUGE-1 by nearly **4 points** over `t5-small` (24.10 $\rightarrow$ 27.90) and ROUGE-L by **3 points** (22.38 $\rightarrow$ 25.37), confirming that instruction-tuned scale is directly translating into better title quality.

Qualitative Highlights: Entity-Aware Titles

Content (truncated)	Actual Title	FLAN-T5 Beam-Search Prediction
the zero day initiative published 13 ivanti endpoint manager vulnerabilities in october 2025...	ivanti epm cve 2026 1603 four months between advisory and patch	zdi released 13 ivanti endpoint manager vulnerabilities in october 2025 cve 2026 1603
for centuries a profound misunderstanding has clouded scripture...bride of christ...	the true bride of christ...spiritual warfare	ekklesia the bride of christ

Key Finding

The model correctly extracts scattered named entities ("*zdi*", "*ivanti endpoint manager*", "*cve 2026 1603*") and captures precise thematic/theological context ("*ekklesia the bride of christ*"), decoded via Beam Search (num_beams=4, no_repeat_ngram_size=2).

Conclusion

Resolving the FP32 numerical-stability issue unlocked training for FLAN-T5-Base without further NaN failures. The resulting model compresses complex technical and philosophical post content into accurate, coherent titles and is saved (with its tokenizer) for direct integration into the automated inference pipeline (`make_predictions.py`).

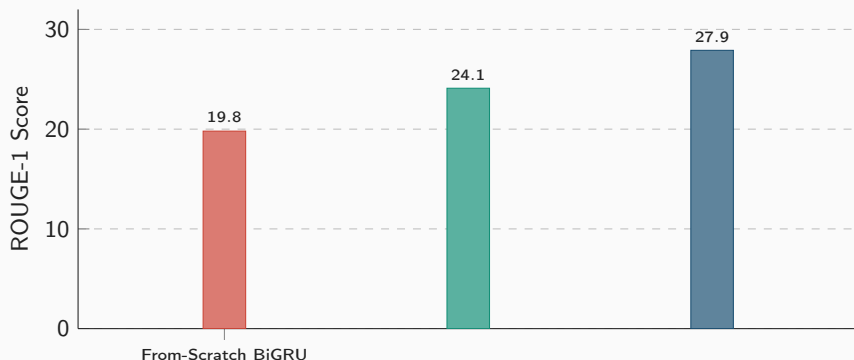
FLAN-T5-Base is selected as the production title-generation model.

Model Comparison & Benchmarking

Head-to-Head: All Three Architectures

Version	Model	BLEU	ROUGE-1	ROUGE-2	ROUGE-L
v1	From-scratch BiGRU + Attention	0.030	19.80	–	–
v2	t5-small (fine-tuned)	–	24.10	9.23	22.38
v3	flan-t5-base (fine-tuned, final)	–	27.90	11.40	25.37

BLEU/ROUGE-1 for v1 reported on the 0–1 scale (0.198); rescaled $\times 100$ above for direct comparability with v2/v3 ROUGE percentages.



Key Finding

Each iteration delivers a monotonic ROUGE-1 improvement – moving from a hand-built recurrent model, to a small pre-trained Transformer, to a larger instruction-tuned Transformer – validating the three-stage experimental strategy.

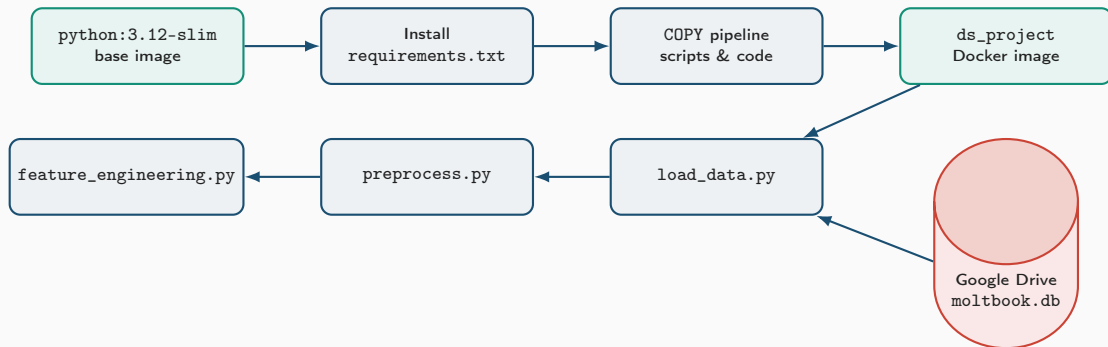
Production Deployment: Docker & Kubernetes

- The full offline pipeline (`load_data.py` → `preprocess.py` → `feature_engineering.py`) is packaged into a **Docker image** so it runs identically on any machine.
- Base image: `python:3.12-slim`
- Dependencies pinned in `requirements.txt`, installed via `pip install -no-cache-dir`
- The raw `moltbook.db` SQLite database is **not** baked into the image – it is downloaded at container start time from Google Drive, keeping the image small and the data source centrally versioned.

Dockerfile structure

1. `FROM python:3.12-slim`
2. `WORKDIR /app`
3. `COPY requirements.txt .`
4. `RUN pip install -no-cache-dir -r requirements.txt`
5. `COPY . .`

Containerization Architecture



docker run triggers: dataset download → load → preprocess (JSON flattening, dedup) → feature engineering → persisted, cleaned artifacts.

Docker Build in Practice

```
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> minikube start
* minikube v1.38.1 on Microsoft Windows 11 Enterprise 25H2
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Restarting existing docker container for "minikube" ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> minikube docker-env | Invoke-Expression
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> docker build -t ds-pipeline:latest .
[+] Building 75.4s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 489B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim 2.2s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:d764629ce0dd 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 1.70kB                                   0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> [3/5] COPY requirements_docker.txt .                            0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements_docker.txt 71.6s
=> [5/5] COPY . .                                                  0.2s
=> exporting to image                                              1.2s
=> => exporting layers                                              1.1s
=> => writing image sha256:c84612274f418d6db93636d9539a969be50e16f74 0.0s
=> => naming to docker.io/library/ds-pipeline:latest              0.0s
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> |
```

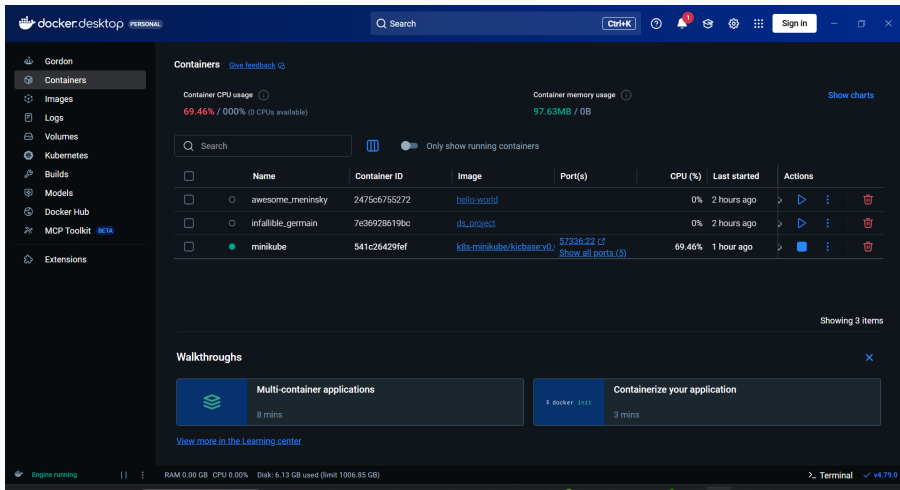
`docker build -t ds_project .` completes in 359.3s across 10 cached/executed layers – base image pull, dependency installation, code copy, and final image export.

Container Execution: Data Download & Pipeline Kickoff

```
=> => exporting config sha256:67f640c5f2eba9ef71a8bc2e27dab0dc3e74c0207f1f6c8 0.0s
=> => exporting attestation manifest sha256:88c7c1781e5948ed663d4df6628702883 0.0s
=> => exporting manifest list sha256:889d9ea1964dbef6763f8d186799114f289cd4e3 0.0s
=> => naming to docker.io/library/ds_project:latest 0.0s
=> => unpacking to docker.io/library/ds_project:latest 6.6s
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> docker run ds_project
Downloading...
From (original): https://drive.google.com/uc?id=147RZEz8WE3CTBR_zpDMdzMG447ZeTaf
From (redirected): https://drive.google.com/uc?id=147RZEz8WE3CTBR_zpDMdzMG447ZeTaf&confirm=t&uuiid=7ed2b68c-95c6-4bca-a8d7-28656d87a986
To: /app/database/downloaded_file.tmp
100%|██████████| 284M/284M [03:06<00:00, 1.52MB/s]
[PROCESS] Loading data from database...
[SUCCESS] Data connection verified and ready for preprocessing.
[INFO] Connecting to database: /app/scripts/./database/moltbook.db
[INFO] Base tables loaded successfully.
[INFO] Flattening JSON column: 'data'
[INFO] Extracted 25 numerical/categorical columns.
[INFO] Flattening JSON column: 'data'
[INFO] Extracted 32 numerical/categorical columns.
[INFO] Flattening JSON column: 'data'
[INFO] Extracted 26 numerical/categorical columns.
[INFO] Extracting agent features from relational tables...
[INFO] Merging records and deduplicating agent identifiers...
```

`docker run ds_project` downloads the 284 MB raw database from Google Drive at runtime, verifies the connection, then begins flattening nested JSON columns from the posts, agents, and comments tables (25, 32, and 26 extracted numerical/categorical columns respectively) before merging and deduplicating agent identifiers.

Docker Desktop: Local Container Management



The screenshot shows the Docker Desktop application window. The top bar includes the Docker logo, the name 'Gordon', a search bar, a 'Ctrl+K' button, and a 'Sign in' button. The left sidebar contains navigation links: Gordon, Containers (selected), Images, Logs, Volumes, Kubernetes, Builds, Models, Docker Hub, MCP Toolkit (beta), and Extensions. The main area is titled 'Containers' and shows overall usage: 'Container CPU usage 69.46% / 000% (0 CPUs available)' and 'Container memory usage 97.63MB / 0B'. Below this is a search bar and a toggle for 'Only show running containers'. A table lists three containers:

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	awesome_meninsky	2475c6755272	hello-world		0%	2 hours ago	[Stop] [Refresh] [More]
☐	infallible_germain	7e36928619bc	ds_project		0%	2 hours ago	[Stop] [Refresh] [More]
☐	minikube	541c26429fef	k8s-minikube/kicbase:v0.1	57336:22 [↕] Show all ports (5)	69.46%	1 hour ago	[Stop] [Refresh] [More]

Below the table, it says 'Showing 3 items'. There is a 'Walkthroughs' section with two cards: 'Multi-container applications' (8 mins) and 'Containerize your application' (3 mins). At the bottom, a status bar shows 'Engine running', 'RAM 0.00 GB', 'CPU 0.00%', 'Disk: 6.13 GB used (limit 1006.85 GB)', and a 'Terminal' button with version 'v4.79.0'.

Docker Desktop confirms the minikube container is running (69.46% CPU during active orchestration), alongside the previously built ds_project and a hello-world sanity-check container – giving a visual, real-time confirmation of local infrastructure health.

- Beyond a single `docker run`, the pipeline is orchestrated as a **Kubernetes Job** on a local `minikube` cluster – a step toward production-grade, schedulable, restartable execution.
- Cluster provisioned with `minikube start`: 2 CPUs, 4000 MB memory, Kubernetes **v1.35.1** on Docker **29.2.1**.
- Bridge CNI configured; `storage-provisioner` and `default-storageclass` addons enabled automatically.

Deployment workflow

1. `minikube start`
2. `docker build -t ds-pipeline:latest .` (inside the `minikube` Docker daemon)
3. `kubectl apply -f pipeline-job.yaml`
4. `kubectl get pods` / `get jobs` to monitor
5. `kubectl logs job/ds-pipeline-job` for the execution trace

Kubernetes Cluster Health Check

```
* Creating docker container (CPUs=2, Memory=4000MB) ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\Users\user> kubectl get nodes
NAME          STATUS    ROLES          AGE    VERSION
minikube      Ready     control-plane   3m1s   v1.35.1
PS C:\Users\user> kubectl get pods -A
NAMESPACE     NAME                                READY   STATUS    RESTARTS
              AGE
kube-system   coredns-7d764666f9-85cq5           1/1     Running   0
              3m36s
kube-system   etcd-minikube                       1/1     Running   0
              3m43s
kube-system   kube-apiserver-minikube             1/1     Running   0
              3m43s
kube-system   kube-controller-manager-minikube    1/1     Running   0
              3m42s
kube-system   kube-proxy-pf777                    1/1     Running   0
              3m37s
kube-system   kube-scheduler-minikube             1/1     Running   0
              3m43s
kube-system   storage-provisioner                 1/1     Running   1 (3m14s
              ago) 3m40s
PS C:\Users\user> |
```

`kubectl get nodes` confirms the minikube control-plane node is Ready; `kubectl get pods -A` shows all core kube-system components (coredns, etcd, kube-apiserver, kube-controller-manager, kube-proxy, kube-scheduler, storage-provisioner) in a healthy Running state before the pipeline Job is scheduled.

End-to-End Kubernetes Job Execution

```
stderr:
failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine; check if the path is correct and if the daemon is running: open //./pipe/dockerDesktopLinuxEngine: The system cannot find the
file specified.
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> minikube start
* minikube v1.38.1 on Microsoft Windows 11 Enterprise 25H2
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Restarting existing docker container for "minikube" ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Verifying Kubernetes components...
  - Using image get.io/kas-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubect1 is now configured to use "minikube" cluster and "default" namespace by default
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> minikube docker-env | Invoke-Expression
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> docker build --t ds-pipeline:latest .
[+] Building 75.4s (10/10) FINISHED          docker:default
=> [internal] load build definition from Dockerfile          0.0s
=> => transferring dockerfile: 489B                          0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim 2.2s
=> [internal] load .dockerignore                             0.0s
=> => transferring context: 2B                                  0.0s
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:d764629ce0dd 0.0s
=> [internal] load build context                             0.0s
=> => transferring context: 1.70kB                             0.0s
=> [2/5] WORKDIR /app                                       0.0s
=> [3/5] COPY requirements_docker.txt .                     0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements_docker.txt 71.6s
=> [5/5] COPY . .                                           0.2s
=> => exporting to image                                       1.2s
=> => exporting layers                                         1.1s
=> => writing image sha256:c84612274f618d5db936dd9539a969be50e16f74 0.0s
=> => naming to docker.io/library/ds-pipeline:latest         0.0s
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> notepad pipeline-job.yaml
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> kubect1 apply -f pipeline-job.yaml
job.batch/ds-pipeline-job created
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> kubect1 get pods
NAME                                READY    STATUS    RESTARTS   AGE
ds-pipeline-job-2z64p              1/1      Running   0           13s
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> kubect1 get jobs
NAME                                STATUS    COMPLETIONS   DURATION   AGE
ds-pipeline-job                    Running   0/1            33s        33s
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> kubect1 get pods
NAME                                READY    STATUS    RESTARTS   AGE
ds-pipeline-job-2z64p              0/1      Complete   0           6m48s
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> kubect1 get jobs
NAME                                STATUS    COMPLETIONS   DURATION   AGE
ds-pipeline-job                    Complete 1/1           5m16s      6m57s
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> kubect1 logs job/ds-pipeline-job
Downloading...
From (original): https://drive.google.com/uc?fid=147R2Ez8WE3CT8R_zpDwmdzMG447ZeTaf
From (redirected): https://drive.google.com/uc?fid=147R2Ez8WE3CT8R_zpDwmdzMG447ZeTaf&confirm=t&uid=481e69ba-5588-4809-87db-c3fa2c4ca544
```

docker build ds-pipeline:latest (75.4s, leveraging cached layers) → kubect1 apply -f pipeline-job.yaml → Job transitions Running → Complete (duration ~5m16s) → kubect1 logs streams the full pipeline trace directly from the pod.

Full Pipeline Execution Log (Inside the Pod)

```
ds-pipeline-job completed 1/1
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> kubectl logs job/ds-pipeline-job
Downloading...
From (original): https://drive.google.com/uc?id=147RZEz8WE3CT8R_zpDKmdzKG447ZeTaf
From (redirected): https://drive.google.com/uc?id=147RZEz8WE3CT8R_zpDKmdzKG447ZeTaf&confirm=t&uuiid=481e69ba-5588-4809-87b0-c3fa2c4ca544
To: /app/database/downloaded_file.tmp
100% [██████████] 284M/284M [04:58<00:00, 950kB/s]
[PROCESS] Loading data from database...
[SUCCESS] Data connection verified and ready for preprocessing.
[INFO] Connecting to database: /app/scripts/../database/moltbook.db
[INFO] Base tables loaded successfully.
[INFO] Flattening JSON column: 'data'
[INFO] Extracted 25 numerical/categorical columns.
[INFO] Flattening JSON column: 'data'
[INFO] Extracted 32 numerical/categorical columns.
[INFO] Flattening JSON column: 'data'
[INFO] Extracted 26 numerical/categorical columns.
[INFO] Extracting agent features from relational tables...
[INFO] Merging records and deduplicating agent identifiers...
[INFO] Agents table finalized: 6385 unique records.
[INFO] Dropping redundant features for schema normalization...
[INFO] Database schema successfully normalized.
[INFO] Persisting cleaned dataset to SQLite: /app/scripts/../database/moltbook_clean.db
[INFO] Exporting relational tables to CSV format...
[SUCCESS] Data preprocessing operations completed.
[INFO] Connecting to database for network edge extraction...
[INFO] Constructing network edges based on user interactions...
[INFO] Edge extraction completed. Valid interactions found: 13511
[INFO] Initializing NetworkX Directed Graph...
[SUCCESS] Network topology serialized and saved to: /app/scripts/../database/processed/moltbook_interaction_graph.graphml
-----
[INFO] Initiating Data Science Processing Pipeline (Real Execution)
-----
[INFO] Database not found locally. Downloading from Google Drive...
[SUCCESS] File downloaded from Drive.
[INFO] Raw database file detected. Renaming...
[SUCCESS] Database ready for processing.
[PROCESS] Executing: scripts/load_data.py...
[SUCCESS] scripts/load_data.py completed in 0.43 seconds.

[PROCESS] Executing: scripts/preprocess.py...
[SUCCESS] scripts/preprocess.py completed in 8.84 seconds.

[PROCESS] Executing: scripts/feature_engineering.py...
[SUCCESS] scripts/feature_engineering.py completed in 0.88 seconds.

-----
[INFO] Pipeline execution completed successfully.
-----
PS E:\ElecEng vol 8\Data Science\project\Moltbook-Agent-Analysis-main> |
```

Reading the Pipeline Log: What Actually Happened

- Database auto-downloaded from Google Drive (284 MB), renamed, and verified before processing.
- JSON columns flattened across all three raw tables (25 / 32 / 26 columns extracted).
- Agent features merged and deduplicated → **6,385 unique agent records** finalized.
- Schema normalized; cleaned dataset persisted to `moltbook_clean.db` and exported to CSV.

Timing breakdown

- `load_data.py`: 0.43s
- `preprocess.py`: 8.84s
- `feature_engineering.py`: 0.88s

Key Finding

The pipeline also constructs a **directed interaction graph** (NetworkX) from user interactions, discovering **13,511** valid edges, serialized to `moltbook_interaction_graph.graphml` for downstream network analysis.

Reproducibility

Anyone on the team (or a grader) can run `docker build && docker run` and get byte-identical preprocessing results – no “works on my machine” dependency drift.

Portability & Scalability

Running the same image as a Kubernetes `Job` demonstrates a direct path from local prototyping to a schedulable, restartable, horizontally-scalable production workflow.

Separation of Data and Code

The image itself stays lightweight; the raw database is fetched at runtime from a single, centrally-managed source (Google Drive), so the source-of-truth data can be updated without rebuilding the image.

Executive Summary

1. **From-scratch RNN baseline:** proved the mechanics (custom vocabulary, Bahdanau attention, BiGRU) but collapsed into catastrophic repetition (BLEU 0.03, ROUGE-1 19.8) – confirming that recurrent architectures without pre-training cannot solve this task well.
2. **T5-Small transfer learning:** an immediate, large jump in quality (ROUGE-1 24.10) simply by fine-tuning a small pre-trained Transformer; Beam Search further improved coherence with zero retraining.
3. **FLAN-T5-Base (final model):** the best result (ROUGE-1 27.90, ROUGE-L 25.37), after correctly diagnosing and fixing an `fp16` NaN-loss instability by switching to FP32 training.
4. **Every experiment tracked in MLflow**, and the final model + tokenizer saved for direct production inference.

1. The complete data pipeline is **containerized** with Docker (`python:3.12-slim` base, pinned dependencies, runtime data download).
2. The same image is **orchestrated on Kubernetes** via `minikube`, running as a monitorable, restartable Job – validated end to end with `kubectl get nodes/pods/jobs` and `kubectl logs`.
3. The pipeline log confirms full, successful execution: data ingestion, JSON flattening, agent deduplication (6,385 unique agents), schema normalization, and interaction-graph construction (13,511 edges).

From an idea to a deployed, benchmarked ML system

This work took the Moltbook title-generation problem from a from-scratch RNN experiment, through two successive Transformer fine-tuning stages, to a final FLAN-T5-Base model with strong, well-documented ROUGE performance – and then packaged the entire data pipeline into a reproducible, Kubernetes-orchestrated Docker deployment.

- Every architectural decision, failure, and fix is **documented and justified** – from the Bahdanau attention sanity check to the `fp16` NaN-loss diagnosis.
- The final model (ROUGE-1 27.90) and the containerized pipeline are both ready for downstream integration.

Thank you